

Assignment 4 - Scheduling

May 7, 2025

1 Revision History

- 2025-05-02: Initial release.
- 2025-05-07: Modified the content of `schedtest`, it now uses `ticks` as the time reference, and the test conditions are stricter.

Modified `Makefile`, it now includes the `handin` target.

The following files have been modified:

- `user/src/schedtest.c`
- `os/ktest/ktest_syscall.c`, `os/ktest/ktest.h`
- `Makefile`

2 Background

Currently, xv6 uses the Round Robin (RR) scheduling algorithm. Each time a scheduling decision is needed, the old task is placed at the end of the queue, and the task to be executed next (i.e., the process) is taken from the front of the queue. We learned in the theoretical lectures that scheduling algorithms like RR have the concept of a time slice, which represents how many time units a process can execute. In the kernel, each time unit is the time between two interrupts. In the current xv6 RR algorithm, the length of each time slice is the interval between clock interrupts.

This assignment requires you to implement a scheduling algorithm on xv6 that:

1. Allows the length of the time slice to be greater than 1, meaning the time slice length of a process can be N clock interrupt intervals. This value is a property of each process, i.e., different processes can have different time slice lengths.
2. Implements Priority Scheduling, where higher priority processes get longer time slices each time they execute.
3. Prints the scheduling information of a process after it exits, including Waiting Time and Turnaround Time, etc.

Build commit hash:

3 Hints

Each time a clock interrupt occurs (`usertrap()`), the kernel uses `yield()` to relinquish the CPU. This method returns CPU control to the scheduler; the scheduler will place it at the end of the Ready Queue and then select the process at the head of the Ready Queue to execute.

When the next process is selected, it either starts executing the code from `first_sched_ret` when it is scheduled for the first time, or it returns from the `yield` in the previous `usertrap`. In this case, the call chain is `usertrap -> yield -> sched -> swtch --- (to scheduler) --- (go back from scheduler) -- swtch returns -> sched returns -> yield returns`. In summary, the selected process will eventually call `usertrapret` to return to user space.

When creating a process, xv6 allocates a `struct proc` structure for each process, which contains all the information about that process. You need to add some member variables to this structure to store the scheduling-related information of the process.

In xv6, process scheduling is implemented by the `scheduler()` function in `sched.c`; switching to the scheduler is implemented by the `sched()` function. You may need to modify these functions to complete this assignment.

4 Specifications

You first need to implement the functionality that allows the time slice length to be greater than 1.

You can add a member variable to `struct proc` to store the available time slice for each process. Each time before switching processes, decrement the available time slice. Only when the remaining time slice is 0, actually switch the process and reset its available time slice to `FULL_QUANTUM` (a macro defined in `defs.h`).

Then, implement the `setpriority` system call.

The `setpriority` system call takes an integer argument representing the priority of the process, with a value range of $[0, 10)$. A lower numerical value indicates a higher priority; that is, when resetting the available time slice, a higher priority process will have a larger available time slice. [We recommend using the following expression to calculate the available time slice: `FULL\_QUANTUM - priority \* 2`.](#)

The syscall handling code has already been added for you in `syscall_ids.h`, `syscall.c`, and `user/syscall.h`. You need to implement the specific logic of this system call in `sched.c`.

5 Checkpoints

This assignment is worth a total of 4 points, with 1 Checkpoint and a report. Passing the single Checkpoint is worth 4 points. The report is not graded but is mandatory. Your report should meet the following requirements:

- Report in PDF file format.
- Screenshots of each checkpoint running successfully.
- You are not required to answer the thinking questions raised in the guidelines in the report, but we encourage you to think about and answer them.

- Record the total time spent completing this assignment.
- (Optional) Describe the difficulties you encountered while completing this assignment, or describe what additional guidance you think this assignment needs.
- (Optional) You can share your ideas for completing this assignment in the report.
- The more concise the better.

Requirements You should use `make runsmp` to start **multi-core** xv6. For this assignment, we provide `usertest`. After the kernel starts, you should be able to see a `schedtest` item after the `applist:` prompt. Stable passing is considered passing the Checkpoint. The Time and other information you eventually print may vary each time, but you only need to ensure they are reasonable (higher priority processes execute faster).

If the test passes, you should see output similar to the following:

```
1 ===
2 schedtest: 10-times tests passed
3 ===
```

Checkpoint Description This Checkpoint will fork 5 processes and call `setpriority` to set their priorities. Processes with lower priority values have higher priority. Higher priority processes should take less time to complete the same task (worker).

6 Submission

Rename your report to `report.pdf`, place it in the same directory as this PDF file, and run `make handin`. This will generate a `handin.zip` compressed file. Upload this file to Blackboard.